01 LC 215 | Kth Largest Element in an Array (Medium)

Problem: Given an integer array nums and an integer k, return the kth largest element in the array. Note that it is the kth largest element in the sorted order, not the kth distinct element. Can you solve it without sorting?

Constraints: 1 <= k <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int findKthLargest(vector<int>& nums, int k) {
    array<int, 20001> count{};
    for (int x : nums) ++count[x + 10000];
    for (int i = 20000; i >= 0; --i) {
      k -= count[i];
      if (k <= 0) return i - 10000;
    }
    return 0;
  }
};
```

17 LC 2402 | Meeting Rooms III (Hard)

Problem: Assign each half-open meeting [start,end) to the lowest-numbered available room. If all rooms are busy, delay it until a room frees, preserving duration and priority by original start time. Count meetings per room; return the most-used room, breaking ties by smallest number.

Constraints: 1 <= n <= 100; 1 <= meetings.length <= 10^5; meetings[i].length == 2; 0 <= start_i < end_i <= 5 * 10^5; All the values of start_i are unique.

```
class Solution {
public:
  int mostBooked(int n, vector<vector<int>>&
    meetings) {
    ranges::sort(meetings);
    using pli = pair<long long, int>;
    priority_queue<pli, vector<pli>, greater<pli>>
      busy;
    priority_queue<int, vector<int>, greater<int>>
      idle;
    for (int i = 0; i < n; ++i) {
      idle.push(i);
    }
    vector<int> cnt(n);
    for (auto& v : meetings) {
      int s = v[0], e = v[1];
      while (!busy.empty() && busy.top().first <=
        s) {
        idle.push(busy.top().second);
        busy.pop();
      }
      int i = 0;
      if (!idle.empty()) {
        i = idle.top();
        idle.pop();
        busy.push({e, i});
      } else {
        auto x = busy.top();
        busy.pop();
        i = x.second;
        busy.push({x.first + e - s, i});
      }
      ++cnt[i];
    }
    int ans = 0;
    for (int i = 0; i < n; ++i) {
      if (cnt[ans] < cnt[i]) {
        ans = i;
      }
    }
    return ans;
  }
};
```

02 LC 347 | Top K Frequent Elements (Medium)

Problem: Return the k most frequent distinct values in nums, in any order. The answer set is unique. Required runtime is better than O(n log n).

Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; k is in the range [1, the number of unique elements in the array].; It is guaranteed that the answer is unique.

```
class Solution {
public:
  vector<int> topKFrequent(vector<int>& nums, int
    k) {
    unordered_map<int, int> frequency;
    for (int x : nums) ++frequency[x];
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto [value, count] : frequency)
      buckets[count].push_back(value);
    vector<int> answer;
    for (int count = nums.size(); count > 0;
      --count)
      for (int value : buckets[count]) {
        answer.push_back(value);
        if (static_cast<int>(answer.size()) == k)
          return answer;
      }
    return answer;
  }
};
```

08 LC 1046 | Last Stone Weight (Easy)

Problem: Repeatedly smash the two heaviest stones. Equal weights both vanish; unequal weights leave a stone with their difference. Return the final remaining weight, or 0 if no stone remains.

Constraints: 1 <= stones.length <= 30; 1 <= stones[i] <= 1000

```
class Solution {
public:
  int lastStoneWeight(vector<int>& stones) {
    priority_queue<int> pq;
    for (int x : stones) {
      pq.push(x);
    }
    while (pq.size() > 1) {
      int y = pq.top();
      pq.pop();
      int x = pq.top();
      pq.pop();
      if (x != y) {
        pq.push(y - x);
      }
    }
    return pq.empty() ? 0 : pq.top();
  }
};
```

49 LC 2558 | Take Gifts From the Richest Pile (Easy)

Problem: Exactly k times, choose a pile with the most gifts and leave floor(sqrt(pile)) gifts there. Return the number of gifts remaining across all piles.

Constraints: 1 <= gifts.length <= 10^3; 1 <= gifts[i] <= 10^9; 1 <= k <= 10^3

```
class Solution {
public:
  long long pickGifts(vector<int>& gifts, int k) {
    make_heap(gifts.begin(), gifts.end());
    while (k--) {
      pop_heap(gifts.begin(), gifts.end());
      gifts.back() = sqrt(gifts.back());
      push_heap(gifts.begin(), gifts.end());
    }
    return accumulate(gifts.begin(), gifts.end(),
      0LL);
  }
};
```

03 LC 23 | Merge k Sorted Lists (Hard)

Problem: You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Constraints: k == lists.length; 0 <= k <= 10^4; 0 <= lists[i].length <= 500; -10^4 <= lists[i][j] <= 10^4; lists[i] is sorted in ascending order.; The sum of lists[i].length will not exceed 10^4.

```
class Solution {
public:
  ListNode* mergeKLists(vector<ListNode*>& lists)
    {
    auto cmp = [](ListNode* a, ListNode* b) {
      return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>,
      decltype(cmp)> pq;
    for (auto head : lists) {
      if (head) {
        pq.push(head);
      }
    }
    ListNode* dummy = new ListNode();
    ListNode* cur = dummy;
    while (!pq.empty()) {
      ListNode* node = pq.top();
      pq.pop();
      if (node->next) {
        pq.push(node->next);
      }
      cur->next = node;
      cur = cur->next;
    }
    return dummy->next;
  }
};
```

30 LC 2353 | Design a Food Rating System (Medium)

Problem: Implement FoodRatings(foods,cuisines,ratings), changeRating(food,newRating), and highestRated(cuisine). Return the food with the greatest current rating in that cuisine, breaking ties lexicographically. Food names/cuisines do not change.

Constraints: 1 <= n <= 2 * 10^4; n == foods.length == cuisines.length == ratings.length; 1 <= foods[i].length, cuisines[i].length <= 10; foods[i], cuisines[i] consist of lowercase English letters.; 1 <= ratings[i] <= 10^8; All the strings in foods are distinct.; food will be the name of a food item in the system across all calls to changeRating.; cuisine will be a type of cuisine of at least one food item in the system across all calls to highestRated.; At most 2 * 10^4 calls in total will be made to changeRating and highestRated.

```
class FoodRatings {
public:
  FoodRatings(vector<string>& foods,
    vector<string>& cuisines, vector<int>&
    ratings) {
    for (int i = 0; i < foods.size(); ++i) {
      string food = foods[i], cuisine =
        cuisines[i];
      int rating = ratings[i];
      d[cuisine].insert({-rating, food});
      g[food] = {rating, cuisine};
    }
  }
  void changeRating(string food, int newRating) {
    auto [oldRating, cuisine] = g[food];
    g[food] = {newRating, cuisine};
    d[cuisine].erase({-oldRating, food});
    d[cuisine].insert({-newRating, food});
  }
  string highestRated(string cuisine) {
    return d[cuisine].begin()->second;
  }
private:
  unordered_map<string, set<pair<int, string>>> d;
  unordered_map<string, pair<int, string>> g;
};
```

04 LC 295 | Find Median from Data Stream (Hard)

Problem: Implement MedianFinder with addNum(num) and findMedian(). For an odd number of inserted values return the middle value; for an even number return the average of the two middle values. Median queries occur only after insertion.

Constraints: -10^5 <= num <= 10^5; There will be at least one element in the data structure before calling findMedian.; At most 5 * 10^4 calls will be made to addNum and findMedian.

```
class MedianFinder {
public:
  MedianFinder() {
  }
  void addNum(int num) {
    maxQ.push(num);
    minQ.push(maxQ.top());
    maxQ.pop();
    if (minQ.size() > maxQ.size() + 1) {
      maxQ.push(minQ.top());
      minQ.pop();
    }
  }
  double findMedian() {
    return minQ.size() == maxQ.size() ?
      (minQ.top() + maxQ.top()) / 2.0 :
      minQ.top();
  }
private:
  priority_queue<int> maxQ;
  priority_queue<int, vector<int>, greater<int>>
    minQ;
};
```

10 LC 378 | Kth Smallest Element in a Sorted Matrix (Medium)

Problem: Given an n x n matrix where each of the rows and columns is sorted in ascending order, return the kth smallest element in the matrix. Note that it is the kth smallest element in the sorted order, not the kth distinct element. You must find a solution with a memory complexity better than O(n^2).

Constraints: n == matrix.length == matrix[i].length; 1 <= n <= 300; -10^9 <= matrix[i][j] <= 10^9; All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.; 1 <= k <= n^2

```
class Solution {
public:
  int kthSmallest(vector<vector<int>>& matrix, int
    k) {
    int n = matrix.size();
    int left = matrix[0][0], right = matrix[n -
      1][n - 1];
    while (left < right) {
      int mid = left + right >> 1;
      if (check(matrix, mid, k, n)) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }
private:
  bool check(vector<vector<int>>& matrix, int mid,
    int k, int n) {
    int count = 0;
    int i = n - 1, j = 0;
    while (i >= 0 && j < n) {
      if (matrix[i][j] <= mid) {
        count += (i + 1);
        ++j;
      } else {
        --i;
      }
    }
    return count >= k;
  }
};
```

05 LC 703 | Kth Largest Element in a Stream (Easy)

Problem: Implement KthLargest(k,nums) with add(val), which inserts val and returns the kth largest value seen, counting duplicate occurrences. Each add query is guaranteed to have at least k values available.
Constraints: 0 <= nums.length <= 10^4; 1 <= k <= nums.length + 1; -10^4 <= nums[i] <= 10^4; -10^4 <= val <= 10^4; At most 10^4 calls will be made to add.

```
class KthLargest {
public:
  KthLargest(int k, vector<int>& nums) {
    this->k = k;
    for (int x : nums) {
      add(x);
    }
  }
  int add(int val) {
    minQ.push(val);
    if (minQ.size() > k) {
      minQ.pop();
    }
    return minQ.top();
  }
private:
  int k;
  priority_queue<int, vector<int>, greater<int>>
    minQ;
};
```

19 LC 1882 | Process Tasks Using Servers (Medium)

Problem: Task j becomes available at time j and lasts tasks[j]. Process waiting tasks in index order, assigning each to a free server of smallest weight, then smallest index. If none is free, wait until one is. Return each task's server index.
Constraints: servers.length == n; tasks.length == m; 1 <= n, m <= 2 * 10^5; 1 <= servers[i], tasks[j] <= 2 * 10^5

```
class Solution {
public:
  vector<int> assignTasks(vector<int>& servers,
    vector<int>& tasks) {
    using pii = pair<int, int>;
    using arr3 = array<long long, 3>;
    priority_queue<pii, vector<pii>, greater<pii>>
      idle;
    priority_queue<arr3, vector<arr3>,
      greater<arr3>> busy;
    for (int i = 0; i < servers.size(); ++i) {
      idle.push({servers[i], i});
    }
    int m = tasks.size();
    vector<int> ans(m);
    for (int j = 0; j < m; ++j) {
      int t = tasks[j];
      while (!busy.empty() && busy.top()[0] <= j)
        {
        auto [_, s, i] = busy.top();
        busy.pop();
        idle.push({(int) s, (int) i});
      }
      if (!idle.empty()) {
        auto [s, i] = idle.top();
        idle.pop();
        ans[j] = i;
        busy.push({(long long) j + t, s, i});
      } else {
        auto [w, s, i] = busy.top();
        busy.pop();
        ans[j] = (int) i;
        busy.push({w + t, s, i});
      }
    }
    return ans;
  }
};
```

06 LC 973 | K Closest Points to Origin (Medium)

Problem: Given an array of points where points[i] = [x_i, y_i] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0). The distance between two points on the X-Y plane is the Euclidean distance (i.e., √(x_1 - x_2)^2 + (y_1 - y_2)^2). You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).
Constraints: 1 <= k <= points.length <= 10^4; -10^4 <= x_i, y_i <= 10^4

```
class Solution {
public:
  vector<vector<int>>
    kClosest(vector<vector<int>>& points, int k) {
    auto distance = [](const vector<int>& p) {
      return 1LL * p[0] * p[0] + 1LL * p[1] *
      p[1]; };
    if (k < static_cast<int>(points.size()))
      ranges::nth_element(points, points.begin() +
        k, {}, distance);
    return {points.begin(), points.begin() + k};
  }
};
```

33 LC 2642 | Design Graph With Shortest Path Calculator (Medium)

Problem: Implement Graph(n,edges) for a directed weighted graph: addEdge(edge) adds a directed edge, and shortestPath(node1,node2) returns the minimum path cost or -1 if unreachable. Paths from a node to itself have cost 0.
Constraints: 1 <= n <= 100; 0 <= edges.length <= n * (n - 1); edges[i].length == edge.length == 3; 0 <= from_i, to_i, from, to, node1, node2 <= n - 1; 1 <= edgeCost_i, edgeCost <= 10^6; There are no repeated edges and no self-loops in the graph at any point.; At most 100 calls will be made for addEdge.; At most 100 calls will be made for shortestPath.

```
class Graph {
public:
  Graph(int n, vector<vector<int>>& edges) {
    this->n = n;
    g = vector<vector<int>>(n, vector<int>(n,
      inf));
    for (auto& e : edges) {
      int f = e[0], t = e[1], c = e[2];
      g[f][t] = c;
    }
  }
  void addEdge(vector<int> edge) {
    int f = edge[0], t = edge[1], c = edge[2];
    g[f][t] = c;
  }
  int shortestPath(int node1, int node2) {
    vector<bool> vis(n);
    vector<int> dist(n, inf);
    dist[node1] = 0;
    for (int i = 0; i < n; ++i) {
      int t = -1;
      for (int j = 0; j < n; ++j) {
        if (!vis[j] && (t == -1 || dist[t] >
          dist[j])) {
          t = j;
        }
      }
      vis[t] = true;
      for (int j = 0; j < n; ++j) {
        dist[j] = min(dist[j], dist[t] + g[t][j]);
      }
    }
    return dist[node2] >= inf ? -1 : dist[node2];
  }
private:
  vector<vector<int>> g;
  int n;
  const int inf = 1 << 29;
};
```

07 LC 692 | Top K Frequent Words (Medium)

Problem: Given an array of strings words and an integer k, return the k most frequent strings. Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Constraints: 1 <= words.length <= 500; 1 <= words[i].length <= 10; words[i] consists of lowercase English letters.; k is in the range [1, The number of unique words[i]]

```
class Solution {
public:
  vector<string> topKFrequent(vector<string>&
    words, int k) {
    unordered_map<string, int> cnt;
    for (const auto& w : words) {
      ++cnt[w];
    }
    vector<string> ans;
    for (const auto& [w, _] : cnt) {
      ans.push_back(w);
    }
    ranges::sort(ans, [&](const string& a, const
      string& b) {
      return cnt[a] > cnt[b] || (cnt[a] == cnt[b]
        && a < b);
    });
    ans.resize(k);
    return ans;
  }
};
```

35 LC 1514 | Path with Maximum Probability (Medium)

Problem: In an undirected graph with independent edge success probabilities, find a path from start to end maximizing the product of its edge probabilities. Return that probability, or 0 if unreachable.

Constraints: 2 <= n <= 10^4; 0 <= start, end < n; start != end; 0 <= a, b < n; a != b; 0 <= succProb.length == edges.length <= 2*10^4; 0 <= succProb[i] <= 1; There is at most one edge between every two nodes.

```
class Solution {
public:
  double maxProbability(int n,
    vector<vector<int>>& edges, vector<double>&
    succProb, int start_node, int end_node) {
    using pdi = pair<double, int>;
    vector<vector<pdi>> g = vector<vector<pdi>>(n,
      vector<pdi>{});
    for (int i = 0; i < edges.size(); ++i) {
      int a = edges[i][0], b = edges[i][1];
      double p = succProb[i];
      g[a].emplace_back(p, b);
      g[b].emplace_back(p, a);
    }
    vector<double> dist(n);
    dist[start_node] = 1;
    priority_queue<pdi> pq;
    pq.emplace(1, start_node);
    while (!pq.empty()) {
      auto [w, a] = pq.top();
      pq.pop();
      if (dist[a] > w) {
        continue;
      }
      for (auto [p, b] : g[a]) {
        auto nw = w * p;
        if (nw > dist[b]) {
          dist[b] = nw;
          pq.emplace(nw, b);
        }
      }
    }
    return dist[end_node];
  }
};
```

09 LC 373 | Find K Pairs with Smallest Sums (Medium)

Problem: You are given two integer arrays nums1 and nums2 sorted in non-decreasing order and an integer k. Define a pair (u, v) which consists of one element from the first array and one element from the second array. Return the k pairs (u_1, v_1), (u_2, v_2), ..., (u_k, v_k) with the smallest sums.

Constraints: 1 <= nums1.length, nums2.length <= 10^5; -10^9 <= nums1[i], nums2[i] <= 10^9; nums1 and nums2 both are sorted in non-decreasing order.; 1 <= k <= 10^4; k <= nums1.length * nums2.length

```
class Solution {
public:
  vector<vector<int>> kSmallestPairs(vector<int>&
    nums1, vector<int>& nums2, int k) {
    auto cmp = [&nums1, &nums2](const pair<int,
      int>& a, const pair<int, int>& b) {
      return nums1[a.first] + nums2[a.second] >
        nums1[b.first] + nums2[b.second];
    };
    int m = nums1.size();
    int n = nums2.size();
    vector<vector<int>> ans;
    priority_queue<pair<int, int>,
      vector<pair<int, int>>, decltype(cmp)>
      pq(cmp);
    for (int i = 0; i < min(k, m); i++)
      pq.emplace(i, 0);
    while (k-- && !pq.empty()) {
      auto [x, y] = pq.top();
      pq.pop();
      ans.emplace_back( initializer_list<int>{
        nums1[x], nums2[y]});
      if (y + 1 < n)
        pq.emplace(x, y + 1);
    }
    return ans;
  }
};
```

18 LC 1834 | Single-Threaded CPU (Medium)

Problem: Tasks have enqueue and processing times. A single CPU, when idle, chooses an available task with smallest processing time, breaking ties by smallest original index. A running task is uninterrupted. Return the execution order.

Constraints: 1 <= tasks.length <= 10^5; tasks[i] = [enqueueTime_i, processingTime_i]; 1 <= enqueueTime_i, processingTime_i <= 10^9

```
class Solution {
public:
  vector<int> getOrder(vector<vector<int>>& tasks)
    {
    int n = 0;
    for (auto& task : tasks) task.push_back(n++);
    ranges::sort(tasks);
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>>
      q;
    int i = 0;
    long long t = 0;
    vector<int> ans;
    while (!q.empty() || i < n) {
      if (q.empty()) t = max(t, (long long)
        tasks[i][0]);
      while (i < n && tasks[i][0] <= t) {
        q.push({tasks[i][1], tasks[i][2]});
        ++i;
      }
      auto [pt, j] = q.top();
      q.pop();
      ans.push_back(j);
      t += pt;
    }
    return ans;
  }
};
```

11 LC 767 | Reorganize String (Medium)

Problem: Given a string s, rearrange the characters of s so that any two adjacent characters are not the same. Return any possible rearrangement of s or return "" if not possible.
Constraints: 1 <= s.length <= 500; s consists of lowercase English letters.

```
class Solution {
public:
  string reorganizeString(string s) {
    vector<int> cnt(26);
    for (char& c : s) ++cnt[c - 'a'];
    int mx = *max_element(cnt.begin(), cnt.end());
    int n = s.size();
    if (mx > (n + 1) / 2) return "";
    vector<vector<int>> m;
    for (int i = 0; i < 26; ++i) {
      if (cnt[i]) m.push_back({cnt[i], i});
    }
    ranges::sort(m);
    ranges::reverse(m);
    string ans = s;
    int k = 0;
    for (auto& e : m) {
      int v = e[0], i = e[1];
      while (v--) {
        ans[k] = 'a' + i;
        k += 2;
        if (k >= n) k = 1;
      }
    }
    return ans;
  }
};
```

25 LC 1851 | Minimum Interval to Include Each Query (Hard)

Problem: For each query q, return the size right-left+1 of the smallest closed interval containing q, or -1 if none exists. Return results in original query order.
Constraints: 1 <= intervals.length <= 10^5; 1 <= queries.length <= 10^5; intervals[i].length == 2; 1 <= left_i <= right_i <= 10^7; 1 <= queries[j] <= 10^7

```
class Solution {
public:
  vector<int> minInterval(vector<vector<int>>&
    intervals, vector<int>& queries) {
    int n = intervals.size(), m = queries.size();
    ranges::sort(intervals);
    using pii = pair<int, int>;
    vector<pii> qs;
    for (int i = 0; i < m; ++i) {
      qs.emplace_back(queries[i], i);
    }
    ranges::sort(qs);
    vector<int> ans(m, -1);
    priority_queue<pii, vector<pii>, greater<pii>>
      pq;
    int i = 0;
    for (auto& [x, j] : qs) {
      while (i < n && intervals[i][0] <= x) {
        int a = intervals[i][0], b =
          intervals[i][1];
        pq.emplace(b - a + 1, b);
        ++i;
      }
      while (!pq.empty() && pq.top().second < x) {
        pq.pop();
      }
      if (!pq.empty()) {
        ans[j] = pq.top().first;
      }
    }
    return ans;
  }
};
```

12 LC 621 | Task Scheduler (Medium)

Problem: You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label. Return the minimum number of CPU intervals required to complete all tasks.
Constraints: 1 <= tasks.length <= 10^4; tasks[i] is an uppercase English letter.; 0 <= n <= 100

```
class Solution {
public:
  int leastInterval(vector<char>& tasks, int n) {
    vector<int> cnt(26);
    int x = 0;
    for (char c : tasks) {
      c -= 'A';
      ++cnt[c];
      x = max(x, cnt[c]);
    }
    int s = 0;
    for (int v : cnt) {
      s += v == x;
    }
    return max((int) tasks.size(), (x - 1) * (n +
      1) + s);
  }
};
```

16 LC 253 | Meeting Rooms II (Hard)

Problem: Given an array of meeting time intervals intervals where intervals[i] = [start_i, end_i], return the minimum number of conference rooms required.
Constraints: 1 <= intervals.length <= 10^4; 0 <= start_i < end_i <= 10^6

```
class Solution {
public:
  int minMeetingRooms(vector<vector<int>>&
    intervals) {
    map<int, int> d;
    for (const auto& e : intervals) {
      d[e[0]]++;
      d[e[1]]--;
    }
    int ans = 0, s = 0;
    for (auto& [_, v] : d) {
      s += v;
      ans = max(ans, s);
    }
    return ans;
  }
};
```

31 LC 264 | Ugly Number II (Medium)

Problem: An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5. Given an integer n, return the nth ugly number.
Constraints: 1 <= n <= 1690

```
class Solution {
public:
  int nthUglyNumber(int n) {
    vector<int> dp(n);
    dp[0] = 1;
    int p2 = 0, p3 = 0, p5 = 0;
    for (int i = 1; i < n; ++i) {
      long long next2 = 2LL * dp[p2], next3 = 3LL
        * dp[p3], next5 = 5LL * dp[p5];
      dp[i] = min(next2, min(next3, next5));
      if (dp[i] == next2) ++p2;
      if (dp[i] == next3) ++p3;
      if (dp[i] == next5) ++p5;
    }
    return dp[n - 1];
  }
};
```

13 LC 502 | IPO (Hard)

Problem: Choose at most k distinct projects. A project requires at least capital[i] current money and, when completed, adds profits[i] to it; the requirement is not spent. Start with w and maximize final capital.

Constraints: 1 <= k <= 10^5; 0 <= w <= 10^9; n == profits.length; n == capital.length; 1 <= n <= 10^5; 0 <= profits[i] <= 10^4; 0 <= capital[i] <= 10^9

```
using pii = pair<int, int>;
class Solution {
public:
  int findMaximizedCapital(int k, int w,
    vector<int>& profits, vector<int>& capital) {
    priority_queue<pii, vector<pii>, greater<pii>>
      q1;
    int n = profits.size();
    for (int i = 0; i < n; ++i) {
      q1.push({capital[i], profits[i]});
    }
    priority_queue<int> q2;
    while (k--) {
      while (!q1.empty() && q1.top().first <= w) {
        q2.push(q1.top().second);
        q1.pop();
      }
      if (q2.empty()) {
        break;
      }
      w += q2.top();
      q2.pop();
    }
    return w;
  }
};
```

36 LC 1631 | Path With Minimum Effort (Medium)

Problem: Travel from top-left to bottom-right of heights using horizontal/vertical moves. A route's effort is its largest absolute height difference across one step. Return the minimum achievable effort.

Constraints: rows == heights.length; columns == heights[i].length; 1 <= rows, columns <= 100; 1 <= heights[i][j] <= 10^6

```
class Solution {
public:
  int minimumEffortPath(vector<vector<int>>&
    heights) {
    int m = heights.size(), n = heights[0].size();
    vector<vector<int>> dist =
      vector<vector<int>>(m, vector<int>(n,
      0x3f3f3f3f));
    dist[0][0] = 0;
    int dirs[5] = {0, 1, 0, -1, 0};
    using T = tuple<int, int, int>;
    priority_queue<T, vector<T>, greater<T>> pq;
    pq.emplace(0, 0, 0);
    while (!pq.empty()) {
      auto [t, i, j] = pq.top();
      pq.pop();
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x < 0 || x >= m || y < 0 || y >= n) {
          continue;
        }
        int d = max(t, abs(heights[x][y] -
          heights[i][j]));
        if (d < dist[x][y]) {
          dist[x][y] = d;
          pq.emplace(d, x, y);
        }
      }
    }
    return dist[m - 1][n - 1];
  }
};
```

14 LC 630 | Course Schedule III (Hard)

Problem: There are n different online courses numbered from 1 to n. You are given an array courses where courses[i] = [duration_i, lastDay_i] indicate that the ith course should be taken continuously for duration_i days and must be finished before or on lastDay_i. You will start on the 1st day and you cannot take two or more courses simultaneously. Return the maximum number of courses that you can take.

Constraints: 1 <= courses.length <= 10^4; 1 <= duration_i, lastDay_i <= 10^4

```
class Solution {
public:
  int scheduleCourse(vector<vector<int>>& courses)
    {
    ranges::sort(courses, [](const vector<int>& a,
      const vector<int>& b) {
      return a[1] < b[1];
    });
    priority_queue<int> pq;
    int s = 0;
    for (auto& e : courses) {
      int duration = e[0], last = e[1];
      pq.push(duration);
      s += duration;
      while (s > last) {
        s -= pq.top();
        pq.pop();
      }
    }
    return pq.size();
  }
};
```

41 LC 1438 | Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit (Medium)

Problem: You are given an array of integers nums and an integer limit. Return the size of the longest non-empty subarray such that the absolute difference between every pair of elements in this subarray is less than or equal to limit.

Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^9; 0 <= limit <= 10^9

```
class Solution {
public:
  int longestSubarray(vector<int>& nums, int
    limit) {
    deque<int> max_q;
    deque<int> min_q;
    int n = nums.size();
    int l = 0;
    for (int r = 0; r < n; ++r) {
      while (!max_q.empty() && nums[max_q.back()]
        < nums[r]) {
        max_q.pop_back();
      }
      while (!min_q.empty() && nums[min_q.back()]
        > nums[r]) {
        min_q.pop_back();
      }
      max_q.push_back(r);
      min_q.push_back(r);
      if (nums[max_q.front()] -
        nums[min_q.front()] > limit) {
        ++l;
        if (max_q.front() < l) {
          max_q.pop_front();
        }
        if (min_q.front() < l) {
          min_q.pop_front();
        }
      }
    }
    return n - l;
  }
};
```

15 LC 1642 | Furthest Building You Can Reach (Medium)
Problem: Move through buildings in order. Descents cost nothing; an ascent costs its height difference in bricks or one ladder. Given bricks and ladders, return the furthest reachable zero-based building index.
Constraints: 1 <= heights.length <= 10^5; 1 <= heights[i] <= 10^6; 0 <= bricks <= 10^9; 0 <= ladders <= heights.length

```
class Solution {
public:
  int furthestBuilding(vector<int>& heights, int
    bricks, int ladders) {
    priority_queue<int, vector<int>, greater<int>>
      q;
    int n = heights.size();
    for (int i = 0; i < n - 1; ++i) {
      int a = heights[i], b = heights[i + 1];
      int d = b - a;
      if (d > 0) {
        q.push(d);
        if (q.size() > ladders) {
          bricks -= q.top();
          q.pop();
          if (bricks < 0) {
            return i;
          }
        }
      }
    }
    return n - 1;
  }
};
```

26 LC 1942 | The Number of the Smallest Unoccupied Chair (Medium)
Problem: Friends arrive at distinct times and take the smallest-numbered unoccupied chair. They free their chairs when leaving; a chair freed at an arrival time is available immediately. Return targetFriend's chair number.
Constraints: n == times.length; 2 <= n <= 10^4; times[i].length == 2; 1 <= arrival_i < leaving_i <= 10^5; 0 <= targetFriend <= n - 1; Each arrival_i time is distinct.

```
class Solution {
public:
  int smallestChair(vector<vector<int>>& times,
    int targetFriend) {
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>>
      busy;
    priority_queue<int, vector<int>, greater<int>>
      idle;
    int n = times.size();
    for (int i = 0; i < n; ++i) {
      times[i].push_back(i);
      idle.push(i);
    }
    ranges::sort(times);
    for (const auto& e : times) {
      int arrival = e[0], leaving = e[1], i =
        e[2];
      while (!busy.empty() && busy.top().first <=
        arrival) {
        idle.push(busy.top().second);
        busy.pop();
      }
      int j = idle.top();
      if (i == targetFriend) {
        return j;
      }
      idle.pop();
      busy.emplace(leaving, j);
    }
    return -1;
  }
};
```

20 LC 2462 | Total Cost to Hire K Workers (Medium)
Problem: Hire exactly k workers sequentially. Each round considers the first candidates and last candidates remaining workers; choose minimum cost, breaking ties by original index. Overlapping windows count workers once. Remove the chosen worker and return total cost.
Constraints: 1 <= costs.length <= 10^5; 1 <= costs[i] <= 10^5; 1 <= k, candidates <= costs.length

```
class Solution {
public:
  long long totalCost(vector<int>& costs, int k,
    int candidates) {
    int n = costs.size();
    if (candidates * 2 > n) {
      ranges::sort(costs);
      return accumulate(costs.begin(),
        costs.begin() + k, 0LL);
    }
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>>
      pq;
    for (int i = 0; i < candidates; ++i) {
      pq.emplace(costs[i], i);
      pq.emplace(costs[n - i - 1], n - i - 1);
    }
    long long ans = 0;
    int l = candidates, r = n - candidates - 1;
    while (k--) {
      auto [cost, i] = pq.top();
      pq.pop();
      ans += cost;
      if (l > r) {
        continue;
      }
      if (i < l) {
        pq.emplace(costs[l], l);
        ++l;
      } else {
        pq.emplace(costs[r], r);
        --r;
      }
    }
    return ans;
  }
};
```

28 LC 2208 | Minimum Operations to Halve Array Sum (Medium)
Problem: You are given an array nums of positive integers. In one operation, you can choose any number from nums and reduce it to exactly half the number. (Note that you may choose this reduced number in future operations.) Return the minimum number of operations to reduce the sum of nums by at least half.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^7

```
class Solution {
public:
  int halveArray(vector<int>& nums) {
    priority_queue<double> pq;
    double s = 0;
    for (int x : nums) {
      s += x;
      pq.push((double) x);
    }
    s /= 2.0;
    int ans = 0;
    while (s > 0) {
      double t = pq.top() / 2.0;
      pq.pop();
      s -= t;
      pq.push(t);
      ++ans;
    }
    return ans;
  }
};
```

21 LC 2542 | Maximum Subsequence Score (Medium)

Problem: Choose k indices. Their score is the sum of nums1 at those indices multiplied by the minimum nums2 value at the same indices. Return the maximum score.
Constraints: n == nums1.length == nums2.length; 1 <= n <= 10^5; 0 <= nums1[i], nums2[j] <= 10^5; 1 <= k <= n

```
class Solution {
public:
  long long maxScore(vector<int>& nums1,
    vector<int>& nums2, int k) {
    int n = nums1.size();
    vector<pair<int, int>> nums(n);
    for (int i = 0; i < n; ++i) {
      nums[i] = {-nums2[i], nums1[i]};
    }
    ranges::sort(nums);
    priority_queue<int, vector<int>, greater<int>>
      q;
    long long ans = 0, s = 0;
    for (auto& [a, b] : nums) {
      s += b;
      q.push(b);
      if (q.size() == k) {
        ans = max(ans, s * -a);
        s -= q.top();
        q.pop();
      }
    }
    return ans;
  }
};
```

44 LC 659 | Split Array into Consecutive Subsequences (Medium)

Problem: Partition sorted nums into one or more subsequences, each of consecutive integers and length at least three. Use every occurrence exactly once. Return whether such a partition exists.
Constraints: 1 <= nums.length <= 10^4; -1000 <= nums[i] <= 1000; nums is sorted in non-decreasing order.

```
class Solution {
public:
  bool isPossible(vector<int>& nums) {
    unordered_map<int, priority_queue<int,
      vector<int>, greater<int>>> d;
    for (int v : nums) {
      if (d.count(v - 1)) {
        auto& q = d[v - 1];
        d[v].push(q.top() + 1);
        q.pop();
        if (q.empty()) {
          d.erase(v - 1);
        }
      } else {
        d[v].push(1);
      }
    }
    for (auto& [_, v] : d) {
      if (v.top() < 3) {
        return false;
      }
    }
    return true;
  }
};
```

22 LC 857 | Minimum Cost to Hire K Workers (Hard)

Problem: Hire exactly k workers. Every worker must receive at least wage[i], and all selected wages must be proportional to quality[i]. Return the minimum total payment; answers within 10^-5 of the optimum are accepted.
Constraints: n == quality.length == wage.length; 1 <= k <= n <= 10^4; 1 <= quality[i], wage[i] <= 10^4

```
class Solution {
public:
  double mincostToHireWorkers(vector<int>&
    quality, vector<int>& wage, int k) {
    int n = quality.size();
    vector<pair<double, int>> t(n);
    for (int i = 0; i < n; ++i) {
      t[i] = {(double) wage[i] / quality[i],
        quality[i]};
    }
    ranges::sort(t);
    priority_queue<int> pq;
    double ans = 1e18;
    int tot = 0;
    for (auto& [x, q] : t) {
      tot += q;
      pq.push(q);
      if (pq.size() == k) {
        ans = min(ans, tot * x);
        tot -= pq.top();
        pq.pop();
      }
    }
    return ans;
  }
};
```

34 LC 743 | Network Delay Time (Medium)

Problem: In a directed weighted graph, a signal starts at k. Return the earliest time by which every node has received it, or -1 if any node is unreachable.
Constraints: 1 <= k <= n <= 100; 1 <= times.length <= 6000; times[i].length == 3; 1 <= u_i, v_i <= n; u_i != v_i; 0 <= w_i <= 100; All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

```
class Solution {
public:
  int networkDelayTime(vector<vector<int>>& times,
    int n, int k) {
    const int inf = 1 << 29;
    using pii = pair<int, int>;
    vector<vector<pii>> g(n);
    for (auto& edge : times) {
      g[edge[0] - 1].emplace_back(edge[1] - 1,
        edge[2]);
    }
    vector<int> dist(n, inf);
    dist[k - 1] = 0;
    priority_queue<pii, vector<pii>, greater<>>
      pq;
    pq.emplace(0, k - 1);
    while (!pq.empty()) {
      auto [d, u] = pq.top();
      pq.pop();
      if (d > dist[u]) {
        continue;
      }
      for (auto& [v, w] : g[u]) {
        if (dist[v] > dist[u] + w) {
          dist[v] = dist[u] + w;
          pq.emplace(dist[v], v);
        }
      }
    }
    int ans = ranges::max(dist);
    return ans == inf ? -1 : ans;
  }
};
```

23 LC 871 | Minimum Number of Refueling Stops (Hard)

Problem: Drive toward target starting with startFuel, consuming one unit per mile. Each station supplies its listed fuel; the tank is unlimited. Return the fewest refueling stops, or -1 if unreachable. Arriving with zero fuel is allowed.

Constraints: 1 <= target, startFuel <= 10^9; 0 <= stations.length <= 500; 1 <= position_i < position_i+1 < target; 1 <= fuel_i < 10^9

```
class Solution {
public:
  int minRefuelStops(int target, int startFuel,
    vector<vector<int>>& stations) {
    priority_queue<int> pq;
    stations.push_back({target, 0});
    int ans = 0, pre = 0;
    for (const auto& station : stations) {
      int pos = station[0], fuel = station[1];
      int dist = pos - pre;
      startFuel -= dist;
      while (startFuel < 0 && !pq.empty()) {
        startFuel += pq.top();
        pq.pop();
        ++ans;
      }
      if (startFuel < 0) {
        return -1;
      }
      pq.push(fuel);
      pre = pos;
    }
    return ans;
  }
};
```

47 LC 1792 | Maximum Average Pass Ratio (Medium)

Problem: Each class has passed/total students. Assign every extra student, guaranteed to pass, to a class. Each assignment increments both passed and total. Maximize the average of class pass ratios and return it.

Constraints: 1 <= classes.length <= 10^5; classes[i].length == 2; 1 <= pass_i <= total_i <= 10^5; 1 <= extraStudents <= 10^5

```
class Solution {
public:
  double maxAverageRatio(vector<vector<int>>&
    classes, int extraStudents) {
    priority_queue<tuple<double, int, int>> pq;
    for (auto& e : classes) {
      int a = e[0], b = e[1];
      double x = (double) (a + 1) / (b + 1) -
        (double) a / b;
      pq.push({x, a, b});
    }
    while (extraStudents--) {
      auto [_, a, b] = pq.top();
      pq.pop();
      a++;
      b++;
      double x = (double) (a + 1) / (b + 1) -
        (double) a / b;
      pq.push({x, a, b});
    }
    double ans = 0;
    while (pq.size()) {
      auto [_, a, b] = pq.top();
      pq.pop();
      ans += (double) a / b;
    }
    return ans / classes.size();
  }
};
```

24 LC 1353 l Maximum Number of Events That Can Be Attended (Medium)

Problem: You are given an array of events where events[i] = [startDay_i, endDay_i]. Every event i starts at startDay_i_ and ends at endDay_i. You can attend an event i at any day d where startDay_i <= d <= endDay_i. You can only attend one event at any time d. Return the maximum number of events you can attend.

Constraints: 1 <= events.length <= 10^5; events[i].length == 2; 1 <= startDay_i <= endDay_i <= 10^5

```
class Solution {
public:
  int maxEvents(vector<vector<int>>& events) {
    unordered_map<int, vector<int>> g;
    int l = INT_MAX, r = 0;
    for (auto& event : events) {
      int s = event[0], e = event[1];
      g[s].push_back(e);
      l = min(l, s);
      r = max(r, e);
    }
    priority_queue<int, vector<int>, greater<int>>
      pq;
    int ans = 0;
    for (int s = l; s <= r; ++s) {
      while (!pq.empty() && pq.top() < s) {
        pq.pop();
      }
      for (int e : g[s]) {
        pq.push(e);
      }
      if (!pq.empty()) {
        pq.pop();
        ++ans;
      }
    }
    return ans;
  }
};
```

48 LC 2233 | Maximum Product After K Increments (Medium)

Problem: You are given an array of non-negative integers nums and an integer k. In one operation, you may choose any element from nums and increment it by 1. Return the maximum product of nums after at most k operations. Since the answer may be very large, return it modulo 10^9 + 7. Note that you should maximize the product before taking the modulo.

Constraints: 1 <= nums.length, k <= 10^5; 0 <= nums[i] <= 10^6

```
class Solution {
public:
  int maximumProduct(vector<int>& nums, int k) {
    ranges::sort(nums);
    int n = nums.size(), count = 1;
    long long remaining = k, level = nums[0];
    while (count < n) {
      long long cost = (nums[count] - level) *
        count;
      if (cost > remaining) break;
      remaining -= cost;
      level = nums[count++];
    }
    level += remaining / count;
    int extra = remaining % count;
    constexpr long long mod = 1'000'000'007;
    long long answer = 1;
    for (int i = 0; i < n; ++i) {
      long long value = i < count ? level + (i <
        extra) : nums[i];
      answer = answer * value % mod;
    }
    return static_cast<int>(answer);
  }
};
```

27 LC 1962 | Remove Stones to Minimize the Total (Medium)
Problem: Exactly k times, choose a stone pile and remove floor(pile/2) stones from it. A pile may be chosen repeatedly. Return the minimum possible remaining total.
Constraints: 1 <= piles.length <= 10^5; 1 <= piles[i] <= 10^4; 1 <= k <= 10^5

```
class Solution {
public:
  int minStoneSum(vector<int>& piles, int k) {
    priority_queue<int> pq;
    for (int x : piles) {
      pq.push(x);
    }
    while (k--) {
      int x = pq.top();
      pq.pop();
      pq.push(x - x / 2);
    }
    int ans = 0;
    while (!pq.empty()) {
      ans += pq.top();
      pq.pop();
    }
    return ans;
  }
};
```

37 LC 778 | Swim in Rising Water (Hard)
Problem: At time t, grid cells of elevation at most t are passable. Move horizontally/vertically between passable cells with no travel-time cost. Return the earliest t permitting travel from top-left to bottom-right.
Constraints: n == grid.length; n == grid[i].length; 1 <= n <= 50; 0 <= grid[i][j] < n^2; Each value grid[i][j] is unique.

```
class Solution {
public:
  int swimInWater(vector<vector<int>>& grid) {
    int n = grid.size();
    int m = n * n;
    vector<int> p(m);
    iota(p.begin(), p.end(), 0);
    auto find = [&](auto&& find, int x) -> int {
      if (p[x] != x) {
        p[x] = find(find, p[x]);
      }
      return p[x];
    };
    vector<int> hi(m);
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        hi[grid[i][j]] = i * n + j;
      }
    }
    array<int, 5> dirs{-1, 0, 1, 0, -1};
    for (int t = 0; t < m; ++t) {
      int id = hi[t];
      int x = id / n, y = id % n;
      for (int k = 0; k < 4; ++k) {
        int nx = x + dirs[k], ny = y + dirs[k +
          1];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n
          && grid[nx][ny] <= t) {
          int a = find(find, x * n + y);
          int b = find(find, nx * n + ny);
          p[a] = b;
        }
      }
      if (find(find, 0) == find(find, m - 1)) {
        return t;
      }
    }
    return 0;
  }
};
```

29 LC 2336 | Smallest Number in Infinite Set (Medium)
Problem: You have a set which contains all positive integers [1, 2, 3, 4, 5, ...]. Implement the SmallestInfiniteSet class: - SmallestInfiniteSet() Initializes the SmallestInfiniteSet object to contain all positive integers. - int popSmallest() Removes and returns the smallest integer contained in the infinite set. - void addBack(int num) Adds a positive integer num back into the infinite set, if it is not already in the infinite set.
Constraints: 1 <= num <= 1000; At most 1000 calls will be made in total to popSmallest and addBack.

```
class SmallestInfiniteSet {
public:
  SmallestInfiniteSet() {
    for (int i = 1; i <= 1000; ++i) {
      s.insert(i);
    }
  }
  int popSmallest() {
    int x = *s.begin();
    s.erase(s.begin());
    return x;
  }
  void addBack(int num) {
    s.insert(num);
  }
private:
  set<int> s;
};
```

38 LC 407 | Trapping Rain Water II (Hard)
Problem: Given an m x n integer matrix heightMap representing the height of each unit cell in a 2D elevation map, return the volume of water it can trap after raining.
Constraints: m == heightMap.length; n == heightMap[i].length; 1 <= m, n <= 200; 0 <= heightMap[i][j] <= 2 * 10^4

```
class Solution {
public:
  int trapRainWater(vector<vector<int>>&
    heightMap) {
    using tii = tuple<int, int, int>;
    priority_queue<tii, vector<tii>, greater<tii>>
      pq;
    int m = heightMap.size(), n =
      heightMap[0].size();
    vector<vector<char>> vis =
      vector<vector<char>>(m, vector<char>(n, 0));
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (i == 0 || i == m - 1 || j == 0 || j ==
          n - 1) {
          pq.emplace(heightMap[i][j], i, j);
          vis[i][j] = true;
        }
      }
    }
    int ans = 0;
    int dirs[5] = {-1, 0, 1, 0, -1};
    while (!pq.empty()) {
      auto [h, i, j] = pq.top();
      pq.pop();
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          !vis[x][y]) {
          ans += max(0, h - heightMap[x][y]);
          vis[x][y] = true;
          pq.emplace(max(heightMap[x][y], h), x,
            y);
        }
      }
    }
    return ans;
  }
};
```

32 LC 313 | Super Ugly Number (Medium)

Problem: A super ugly number is a positive integer whose prime factors are in the array primes. Given an integer n and an array of integers primes, return the nth super ugly number. The nth super ugly number is guaranteed to fit in a 32-bit signed integer.
Constraints: 1 <= n <= 10^5; 1 <= primes.length <= 100; 2 <= primes[i] <= 1000; primes[i] is guaranteed to be a prime number.; All the values of primes are unique and sorted in ascending order.

```
class Solution {
public:
  int nthSuperUglyNumber(int n, vector<int>&
    primes) {
    priority_queue<int, vector<int>, greater<int>>
      q;
    q.push(1);
    int x = 0;
    while (n--) {
      x = q.top();
      q.pop();
      for (int& k : primes) {
        if (x <= INT_MAX / k) {
          q.push(k * x);
        }
        if (x % k == 0) {
          break;
        }
      }
    }
    return x;
  }
};
```

50 LC 3066 | Minimum Operations to Exceed Threshold Value II (Medium)

Problem: Repeatedly remove the two smallest values x<=y and insert 2*x+y. Return the minimum operations until every value is at least k. The input guarantees this is possible.
Constraints: 2 <= nums.length <= 2 * 10^5; 1 <= nums[i] <= 10^9; 1 <= k <= 10^9; The input is generated such that an answer always exists. That is, after performing some number of operations, all elements of the array are greater than or equal to k.

```
class Solution {
public:
  int minOperations(vector<int>& nums, int k) {
    using ll = long long;
    priority_queue<ll, vector<ll>, greater<ll>>
      pq;
    for (int x : nums) {
      pq.push(x);
    }
    int ans = 0;
    for (; pq.size() > 1 && pq.top() < k; ++ans) {
      ll x = pq.top();
      pq.pop();
      ll y = pq.top();
      pq.pop();
      pq.push(x * 2 + y);
    }
    return ans;
  }
};
```

39 LC 632 | Smallest Range Covering Elements from K Lists (Hard)

Problem: You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists. We define the range [a, b] is smaller than range [c, d] if b - a < d - c or a < c if b - a == d - c.
Constraints: nums.length == k; 1 <= k <= 3500; 1 <= nums[i].length <= 50; -10^5 <= nums[i][j] <= 10^5; nums[i] is sorted in non-decreasing order.

```
class Solution {
public:
  vector<int> smallestRange(vector<vector<int>>&
    nums) {
    int n = 0;
    for (auto& v : nums) n += v.size();
    vector<pair<int, int>> t(n);
    int k = nums.size();
    for (int i = 0, j = 0; i < k; ++i) {
      for (int v : nums[i]) {
        t[j++] = {v, i};
      }
    }
    ranges::sort(t);
    int j = 0;
    unordered_map<int, int> cnt;
    vector<int> ans = {-1000000, 1000000};
    for (int i = 0; i < n; ++i) {
      int b = t[i].first;
      int v = t[i].second;
      ++cnt[v];
      while (cnt.size() == k) {
        int a = t[j].first;
        int w = t[j].second;
        int x = b - a - (ans[1] - ans[0]);
        if (x < 0 || (x == 0 && a < ans[0])) {
          ans[0] = a;
          ans[1] = b;
        }
        if (--cnt[w] == 0) {
          cnt.erase(w);
        }
        ++j;
      }
    }
    return ans;
  }
};
```

46 LC 1167 | Minimum Cost to Connect Sticks (Medium)

Problem: Combine sticks two at a time, paying the sum of their lengths and replacing them with that combined stick. Return the minimum total cost to obtain one stick.
Constraints: 1 <= sticks.length <= 10^4; 1 <= sticks[i] <= 10^4

```
class Solution {
public:
  int connectSticks(vector<int>& sticks) {
    priority_queue<int, vector<int>, greater<int>>
      pq;
    for (auto& x : sticks) {
      pq.push(x);
    }
    int ans = 0;
    while (pq.size() > 1) {
      int x = pq.top();
      pq.pop();
      int y = pq.top();
      pq.pop();
      int z = x + y;
      ans += z;
      pq.push(z);
    }
    return ans;
  }
};
```

40 LC 480 | Sliding Window Median (Hard)
Problem: Return the median of every length-k sliding window in nums, in left-to-right order. For even k, use the mean of the two middle values; answers within 10^-5 of the true values are accepted.
Constraints: 1 <= k <= nums.length <= 10^5; -2^31 <= nums[i] <= 2^31 - 1

```
class Solution {
public:
  vector<double> medianSlidingWindow(vector<int>&
    nums, int k) {
    multiset<int> l, r;
    int n = nums.size();
    vector<double> ans;
    for (int i = 0; i < n; ++i) {
      r.insert(nums[i]);
      l.insert(*r.begin());
      r.erase(r.begin());
      while (l.size() - r.size() > 1) {
        r.insert(*l.rbegin());
        l.erase(prev(l.end()));
      }
      int j = i - k + 1;
      if (j >= 0) {
        ans.push_back(k % 2 ? *l.rbegin() :
          ((double) *l.rbegin() + *r.begin()) /
          2);
        auto it = l.find(nums[j]);
        if (it != l.end()) {
          l.erase(it);
        } else {
          r.erase(r.find(nums[j]));
        }
      }
    }
    return ans;
  }
};
```

45 LC 846 | Hand of Straights (Medium)
Problem: Alice has some number of cards and she wants to rearrange the cards into groups so that each group is of size groupSize, and consists of groupSize consecutive cards. Given an integer array hand where hand[i] is the value written on the ith card and an integer groupSize, return true if she can rearrange the cards, or false otherwise.
Constraints: 1 <= hand.length <= 10^4; 0 <= hand[i] <= 10^9; 1 <= groupSize <= hand.length

```
class Solution {
public:
  bool isNStraightHand(vector<int>& hand, int
    groupSize) {
    if (hand.size() % groupSize) {
      return false;
    }
    ranges::sort(hand);
    unordered_map<int, int> cnt;
    for (int x : hand) {
      ++cnt[x];
    }
    for (int x : hand) {
      if (cnt.contains(x)) {
        for (int y = x; y < x + groupSize; ++y) {
          if (!cnt.contains(y)) {
            return false;
          }
          if (--cnt[y] == 0) {
            cnt.erase(y);
          }
        }
      }
    }
    return true;
  }
};
```

42 LC 239 | Sliding Window Maximum (Hard)
Problem: Slide a window of k consecutive elements across nums, advancing one position at a time. Return the maximum value in each window, in window order.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; 1 <= k <= nums.length

```
class Solution {
public:
  vector<int> maxSlidingWindow(vector<int>& nums,
    int k) {
    deque<int> q;
    vector<int> ans;
    for (int i = 0; i < nums.size(); ++i) {
      if (q.size() && i - q.front() >= k) {
        q.pop_front();
      }
      while (q.size() && nums[q.back()] <=
        nums[i]) {
        q.pop_back();
      }
      q.push_back(i);
      if (i >= k - 1) {
        ans.push_back(nums[q.front()]);
      }
    }
    return ans;
  }
};
```

43 LC 451 | Sort Characters By Frequency (Medium)
Problem: Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them.
Constraints: 1 <= s.length <= 5 * 10^5; s consists of uppercase and lowercase English letters and digits.

```
class Solution {
public:
  string frequencySort(string s) {
    unordered_map<char, int> cnt;
    for (char& c : s) {
      ++cnt[c];
    }
    vector<char> cs;
    for (auto& [c, _] : cnt) {
      cs.push_back(c);
    }
    ranges::sort(cs, [&](char& a, char& b) {
      return cnt[a] > cnt[b];
    });
    string ans;
    for (char& c : cs) {
      ans += string(cnt[c], c);
    }
    return ans;
  }
};
```